



Context Course documentation

Quiz 2: Building and Using Skills ▾



Quiz 2: Building and Using Skills

Copy page



Test your understanding of how to build, test, and use skills following the Agent Skills Specification. Answer all 5 questions.

Question 1: Skill Directory Structure

You're building a dataset validation skill. What should your directory structure look like according to the Agent Skills Specification?

- ☐ Just a single SKILL.md file in your project root
- ☒ SKILL.md in root, scripts/ for helper code, references/ for documentation, assets/ for templates

Correct! Correct! This follows the Agent Skills Specification exactly. SKILL.md is required at the root; scripts/, references/, and assets/ are optional but recommended for organization.

- ☐ Everything in a scripts/ folder with SKILL.md inside it
- ☐ Organize folders by agent type (claude/, codex/, opencode/)

You got all the answers!

Question 2: Required Frontmatter Fields

What are the only required frontmatter fields in a SKILL.md file according to the Agent Skills Specification?

- ☐ name, description, version, and license

Ask HuggingChat



- ☒ name and description

Correct! Correct! The specification requires exactly these two fields. Name identifies the skill; description explains what it does and when to use it. All other fields are optional.

- ☐ All metadata fields (name, version, author, etc.)
- ☐ At least 5 fields including metadata and compatibility

Submit

You got all the answers!

Question 3: Validating Your Skill Locally

After building your skill locally, what's the best next step to validate it?

- ☒ Symlink it into your agent's skill directory, start the agent, and test with realistic prompts

Correct! Correct! The first thing that matters is whether the agent can discover and activate the skill in a real workflow. Local install plus realistic prompts catches trigger and usability problems early.

- ☐ Wait until the skill is shared more broadly before you test it with an agent
- ☐ Run only the helper scripts and skip agent testing
- ☐ Read the SKILL.md manually and assume the agent will behave the same way

Submit

You got all the answers!

Question 4: Helper Scripts vs. Instructions

How should you divide content between SKILL.md instructions and helper scripts?

- ☐ Instructions contain all executable code; scripts are just for reference examples
- ☒ Instructions explain what to do and why; scripts contain runnable code agents can actually execute

Correct! Exactly right! Instructions provide strategy and guidance; scripts provide implementation. This separation makes skills teachable.

Ask HuggingChat



- ☐ Everything goes in scripts; instructions are summaries of script functions
- ☐ It depends on the skill; the specification doesn't define this

Submit**You got all the answers!**

Question 5: Activation Debugging

When a skill doesn't activate reliably, what should you do first?

- ☐ Add more helper scripts immediately, even if the trigger description is vague
- ☒ Tighten the description and retest with both obvious and vague prompts

Correct! Correct! Reliable activation comes from a clear description plus realistic prompt testing. Fix the trigger before adding more complexity.

- ☐ Rename the skill folder without changing the description
- ☐ Turn the skill back into a long one-off prompt for every conversation

Submit**You got all the answers!**

Scoring

Count your correct answers:

- **5/5:** Excellent! You understand skill development and the Agent Skills Specification. You're ready to build skills.
- **4/5:** Good understanding. Review the question you missed before moving forward.
- **3/5 or less:** Review the building and usage lessons before proceeding.

Summary

You've completed Quiz 2! You now understand:

[Ask HuggingChat](#)

- How to organize skills following the Agent Skills Specification
- Which frontmatter fields are required vs. optional
- How to validate a skill locally with a real agent
- How to divide content between instructions and executable scripts
- How to debug activation when a skill doesn't fire

These are the core skills for building and maintaining quality agent skills.

Next Steps

You've completed Unit 1: Agent Skills! You can now:

- Build skills following the Agent Skills Specification
- Use skills with Claude Code, Codex, and other agents
- Validate skills in a real project
- Write clear instructions and helper scripts

Next, we move to **Unit 2: Model Context Protocol (MCP)**, where you'll learn how to integrate external tools and APIs with your code agents.

Ready to dive into MCPs?

`</>` [Update](#) on GitHub

← Building Your First Skill

Introduction to Model Context Protocol →



Ask HuggingChat

